

BREAK INTO AEROSPACE

From Computer Science to Aerospace

Career Transition Guide

Targeted for the 2025–2026 Recruitment Cycle

What This Document Covers: Computer science and software engineering graduates are entering aerospace at an accelerating rate — driven by the sector’s growing need for the skills that CS programmes develop better than any other: algorithm design, machine learning, autonomous systems, signal processing software, simulation, and large-scale data engineering. At the same time, the aerospace sector is among the most demanding environments for software engineering practice that exists, with safety-critical development methodologies (DO-178C), deterministic real-time execution requirements, and regulatory approval processes that differ fundamentally from the software development culture of the technology industry. The result is a transition that has both real advantages and specific, consequential gaps — and the CS engineer who does not understand both will either undervalue their transferable skills or discover their gaps at the worst possible moment: in the interview room or, worse, in a certification review.

The gap between how software is developed in the technology industry and how it is developed in safety-critical aerospace is not merely a question of process overhead. It is a question of what software correctness means. In a web application, a bug that causes a crash is a degraded user experience. In a flight management system certified at DO-178C DAL A, a software failure that causes a loss of safe flight path guidance is a Catastrophic failure condition with a required probability of less than 1×10^{-9} per flight hour. The development practices that produce software acceptable for a cloud application — rapid iteration, dynamic memory allocation, third-party library dependencies, test-driven development to line coverage — do not directly produce software acceptable for DO-178C DAL A. The CS engineer who understands why, and who can describe how their existing software engineering discipline would need to adapt, is ready for aerospace. The CS engineer who has memorised “DO-178C is the safety standard” without understanding what it actually requires is not.

This guide is written for software engineers, computer scientists, and data scientists at all career stages who want to transition into aerospace. It covers four substantially different target populations with overlapping but distinct preparation needs: the CS engineer targeting avionics and safety-critical embedded software (the largest demand sector, requiring the deepest regulatory and process adaptation); the ML and data engineer targeting Earth observation, flight data analytics, or autonomous systems (where modern CS skills are more directly applicable but aerospace domain knowledge is essential); the systems and architecture engineer targeting spacecraft ground segment, UTM, or digital twin development (where the systems engineering context of aerospace is the primary new requirement); and the cybersecurity engineer targeting aviation cybersecurity (a rapidly growing domain with a specific regulatory and threat landscape).

The employer landscape spans every aerospace sector: avionics software suppliers (Collins Aerospace, Honeywell, Thales Avionics, GE Aviation avionics), airframe OEM software groups (Airbus Digital and Connected Aircraft, Boeing Digital Aviation, Embraer Software Technology), spacecraft prime contractors with software-intensive missions (Airbus DS ground segment, Thales Alenia Space, GMV, RHEA Group), eVTOL and autonomous aviation developers (Joby, Archer, Wisk, Wing, Zipline), Earth observation data platforms (Planet Labs, Satellogic, ESA Copernicus Data Space team, USGS EROS), UTM and airspace management technology companies (Frequentis, Garmin Aviation, Airwayz, NASA UTM programme office), and aviation cybersecurity specialists and consultancies (Aviation ISAC, SITA, airlines’ cybersecurity teams, specialist consultancies such as Qdos and Eurocae WG-72 participants).

PART I: WHAT AEROSPACE EMPLOYERS ACTUALLY SEE IN A CS ENGINEER

The Genuine Advantages — What Transfers Directly

CS engineers bring a set of technical skills that aerospace has chronically struggled to develop internally. The algorithmic sophistication — complexity analysis, graph algorithms, search and optimisation, dynamic programming — that CS programmes develop rigorously is directly applicable to route optimisation in flight management systems, conflict resolution in UTM, scheduling in satellite mission operations, and trajectory planning in autonomous UAS. The data engineering discipline — distributed data processing at scale, streaming architectures, time-series databases, data pipeline reliability — is directly applicable to the petabyte-scale Earth observation data processing that commercial remote sensing companies handle daily. The machine learning competence — supervised and unsupervised learning, model selection and validation, production ML systems — is directly applicable to autonomous flight perception systems, predictive maintenance from fleet operational data, synthetic aperture radar target classification, and crop stress detection from multispectral imagery.

The software engineering practices that strong CS engineers bring — clean abstraction, modular design, systematic testing, version control discipline, code review culture — are in higher demand in the aerospace software community than they have ever been, because the aerospace sector is recognising that its traditional software development approach produces software that is safe and certified but often expensive to maintain, difficult to reuse, and slow to evolve. Companies developing next-generation avionics, autonomous flight systems, and spacecraft ground segments are actively looking for CS engineers who can bring modern software engineering discipline into the aerospace context — and who can do so while adapting to the safety and regulatory requirements that the domain imposes.

The specific CS skills that are in highest demand in aerospace, in order of immediacy and breadth of application: real-time systems and embedded programming (for avionics and flight control software), machine learning and computer vision (for EO, autonomous systems, and predictive maintenance), software architecture and systems design (for ground segment, UTM, and digital twin), signal processing algorithm implementation (for GNSS, radar, and sensor fusion), and cybersecurity engineering (for aviation network and avionics security).

The Specific Gaps — What Does Not Transfer Without Work

The safety-critical development methodology gap. This is the most consequential gap for CS engineers targeting avionics and safety-critical embedded software roles. DO-178C requires a development process that is unlike anything in the technology industry: all dynamic memory allocation is prohibited in DAL A and B software (no malloc, no new, no garbage collector — ever, because heap corruption at runtime is an undefined failure mode in a safety-critical system); all software requirements must be formally documented before any code is written (requirements-driven development, not agile iteration — the test cases are derived from the requirements, and the tests must demonstrate coverage of every requirement); structural coverage must be demonstrated at the MC/DC level for DAL A (every condition in every decision must be independently shown to affect the decision outcome); and the design and verification teams must be independent (the engineer who wrote a function cannot be the one who verifies it meets its requirements). None of this maps to the iterative, test-first, continuous-deployment software development culture that CS programmes teach and that modern technology companies practice. The CS engineer who does not understand why these constraints exist — and who treats them as bureaucracy rather than as principled engineering responses to the specific failure modes of safety-critical software that they are designed to prevent — will fail aerospace avionics interviews consistently.

The real-time determinism gap. Modern CS education is overwhelmingly focused on software that runs on a general-purpose operating system with dynamic resource allocation, non-deterministic scheduling, and garbage collection. Aviation avionics software runs in a deterministic execution environment where every task has a fixed computation time budget, memory is statically allocated at compile time, interrupt latency is bounded by analysis, and worst-case execution time (WCET) is demonstrated by analysis and measurement rather than assumed from average performance. The CS engineer who has only developed on Linux or Windows, with dynamic allocation and preemptive scheduling, has no experience with the execution model that safety-critical avionics requires. The ARINC 653 partitioned RTOS (the standard IMA avionics

operating environment) has no scheduler daemon, no dynamic process creation, and no runtime memory management — it is a fundamentally different execution model that the CS engineer must understand before they can contribute to avionics software development.

The hardware and physical domain gap. Many CS engineers — particularly those from software-intensive universities and those who have worked exclusively in web, mobile, cloud, or data engineering — have no experience with hardware interfaces, physical constraints, or the physics of the domain their software will operate in. An avionics software engineer must understand what a sensor's output represents physically (not just that it is a number in a buffer), what latency means for a flight control system (not network latency — the time between a control surface command and the aircraft's physical response), and what happens to software behaviour at -55°C (floating point arithmetic is affected by denormal numbers at extreme temperatures; fixed-point arithmetic is preferred for safety-critical flight control for this and other reasons). An Earth observation software engineer must understand what radiance values represent, why a cloud mask must be applied before any analysis, and what coordinate reference system transformation means for geospatial data. The CS engineer without this physical domain understanding will produce software that is algorithmically correct and physically meaningless.

The verification and validation rigour gap. CS engineers are typically strong at unit testing (many practice TDD) and at integration testing. They are typically not experienced with the aerospace V&V methodology — the formal requirements traceability that links every test case to the requirement it verifies, the structural coverage analysis that demonstrates that every line of code, every branch, and every condition has been exercised by the test suite, and the independence requirements that mandate separation between development and verification personnel. In particular, the concept of MC/DC (Modified Condition/Decision Coverage) — which requires demonstrating that each condition in each decision independently affects the outcome — is not part of standard CS testing education and is specifically tested in aerospace avionics software interviews.

The configuration management gap. Aerospace software configuration management is more rigorous than even the best technology industry CM practice. In DO-178C, every software artifact — source code, object code, test results, review records, problem reports, and change impact analyses — is under formal configuration control with a documented change management process. The specific aerospace CM requirement is bidirectional traceability: every change must be traceable to a problem report or a requirements change, and the impact of the change on existing tests and other software components must be formally assessed. The CS engineer who uses Git well but has not experienced formal aerospace CM — the Configuration Index (SCU listing every configuration item under CM), the Problem Report system, and the change impact analysis — has a process gap that DO-178C compliance requires them to close.

The regulatory and certification vocabulary gap. TSO/ETSO authorisation, type certificate, supplemental type certificate, EASA ETSO-2CXXX, FAA TSO-CXXX, Part-21, DO-178C compliance matrix — these are the regulatory vocabulary of avionics product development, and they have no equivalent in the technology industry. The CS engineer who does not know what a TSO authorisation means, what the compliance matrix documents, or what the relationship is between the software's certification artefacts and the aircraft's type certificate is missing the regulatory context that makes the software development work consequential. The software does not fly until EASA or the FAA accepts the compliance evidence — and understanding what that acceptance requires is as important as understanding how to write the code.

By Career Stage

Recent CS Graduate

The recent CS graduate has the advantage of fresh, strong technical fundamentals and the disadvantage of no work experience to draw on for the specific aerospace application of those fundamentals. The transition gap is primarily contextual and regulatory — the physics is largely in place, the regulatory framework and specific aerospace application methods are not. The preparation task is targeted: close the DO-178C and DO-254 literacy gap, learn the ARINC bus architectures (ARINC 429, ARINC 664/AFDX) that define avionics data exchange, and build familiarity with at least one specific aerospace domain in enough depth to discuss a real application.

Mid-Career Software Engineer (5–15 years)

The mid-career software engineer has production systems experience, architecture judgment, and technical leadership credibility that early-career CS graduates lack. The transition task is to translate this track

record into aerospace terms — specifically, to show that the distributed system they designed, the embedded firmware they developed, or the data pipeline they built would be designed differently in an aerospace context, and to demonstrate that they understand how and why. The mid-career engineer targeting avionics software must specifically engage with DO-178C at the level of someone who understands what changes to their development practice it would require — not someone who has read the standard but someone who has reasoned through its implications for their existing workflow.

Senior CS Engineer / Architect

The senior CS engineer or architect brings technical leadership, system design authority, and possibly significant people management experience. The transition task is to demonstrate aerospace-specific authority: an understanding of the certification framework sufficient to make architectural decisions that reduce certification cost and risk, knowledge of the specific failure modes of safety-critical software systems, and the ability to reason about requirements traceability and verification coverage at a system level. Senior avionics software roles carry significant personal responsibility for the certification evidence package, and the interview will probe whether the candidate has the regulatory maturity to be the technical lead for a DO-178C programme.

Domain Mapping — Where Your CS Background Fits Best

Embedded Systems / Firmware / Real-Time → Avionics Software, Flight Control, FADEC

Low-level C/C++, RTOS familiarity, interrupt handling, hardware abstraction layers — all transfer. The aerospace addition: DO-178C development process (no dynamic allocation, requirements-driven, MC/DC coverage), ARINC 653 partitioned RTOS for IMA platforms, ARINC 429 and AFDX communication protocols, and worst-case execution time analysis. The C programming language is the standard for DO-178C DAL A software — Rust is emerging but not yet dominant; Python, Java, and garbage-collected languages are not used in flight-critical paths.

Machine Learning / Computer Vision → Earth Observation ML, Autonomous Systems, Predictive Maintenance

PyTorch, TensorFlow, CNNs, transformers, object detection, semantic segmentation — these transfer with specific adaptations for geospatial and aerospace data. The additions: understanding of the radiometric and geometric properties of satellite imagery (why satellite images are not natural images and why models trained on ImageNet fail on EO data), autonomous flight perception certification under SC-VTOL-01 and emerging AI assurance frameworks (EASA's AI roadmap, DO-333 supplement to DO-178C for model-based development), and predictive maintenance domain knowledge (aircraft fleet data structures, ACARS message formats, QAR/FDR data schema).

Signal Processing / DSP → GNSS Receivers, Radar, Sonar, Sensor Fusion

FFT implementation, filter design, matched filtering, spectral estimation, correlation — these transfer directly to GNSS receiver design, radar signal processing, and sonar processing. The additions: specific signal structures (GPS L1 C/A PRN code, Galileo E1 OS, ARINC 743A for aviation GNSS), aviation navigation integrity concepts (RAIM, ARAIM, protection level computation), and for radar, the specific waveform design and Doppler processing relevant to the application domain.

Systems Architecture / Backend → Spacecraft Ground Segment, UTM, Digital Twin

Distributed systems, event-driven architecture, message queues, API design, cloud infrastructure — these transfer to spacecraft ground segment and UTM software development with specific domain adaptations. The additions: spacecraft communication protocols (CCSDS standards for space data links and file delivery), mission planning and scheduling algorithms, ICAO-compliant message formats for UTM (NET RID, U-Space message format), and the specific latency and reliability requirements of real-time spacecraft commanding versus analytical ground processing.

Cybersecurity → Aviation Cybersecurity

Penetration testing, network security, vulnerability assessment, secure coding, cryptography — all transfer to aviation cybersecurity with domain-specific additions. The additions: aviation-specific threat modelling (the MITRE ATT&CK for ICS framework applied to avionics networks, the Insider Threat risk to ground segment), ARINC 811 (cybersecurity framework for commercial aviation), DO-326A/ED-202A (airworthiness security process), and the specific regulatory pathway for cybersecurity approval under EASA's Aircraft Cybersecurity Regulations.

Data Engineering / MLOps → EO Data Platforms, Flight Data Analytics, Fleet Performance Monitoring

Apache Spark, dbt, Airflow, Kafka, object storage, data lakes — these transfer to EO data processing and flight data analytics with domain-specific adaptations. The additions: geospatial data formats and libraries (GeoTIFF, NetCDF, STAC, rasterio, GDAL), aviation data formats (ACARS, QAR, RINEX, ADS-B), and domain-specific analysis knowledge (remote sensing radiometric correction for EO, performance parameter derivation from flight data for airline analytics).

PART II: 20 SPECIFIC PREP TASKS FOR THE TRANSITION

FOUNDATIONAL — DO THESE FIRST

1. Understand DO-178C at implementation depth — what it actually requires of software development practice, not just conceptually 8–10 hours

Skill: The safety-critical software standard — the largest and most consequential gap for CS engineers targeting avionics roles

Why the field cares: DO-178C is not a testing standard. It is a development assurance standard — a methodology that determines how software must be developed, not just how it must be tested. Its implications for software development practice are specific, counterintuitive from a modern CS perspective, and non-negotiable: no dynamic memory allocation (every object must be allocated statically at compile time — no `std::vector`, no `new`, no `malloc`, no garbage collector, because heap corruption is an unanalysable runtime failure); no recursive function calls (the stack depth of every call tree must be computable at compile time, which recursion prevents); requirements must be written before code is written (the test cases are derived from requirements and must demonstrate that every requirement is tested, which requires requirements to exist before testing — the opposite of test-driven development where tests precede code); and design and verification must be performed by independent personnel (the person who implemented a function cannot be the only person who verifies it meets its requirements — independence is a structural property of the development organisation). The CS engineer who understands why each of these constraints exists — and who can explain them in terms of the specific failure modes of safety-critical software that they are designed to prevent — is demonstrating DO-178C literacy at the level that avionics software interviews require.

Done signal: You can explain why dynamic memory allocation is prohibited in DO-178C DAL A and B software — specifically what the undefined behaviour is that heap allocation can produce in a long-running safety-critical system (memory fragmentation leading to allocation failure at an unpredictable time, heap corruption from a double-free or buffer overflow that cannot be detected at compile time and whose runtime consequence is unpredictable). You can explain what MC/DC coverage requires — giving a specific example of a two-condition decision and demonstrating what test cases are needed to achieve MC/DC versus branch coverage. You can describe what the independence requirement means organisationally — how a company structures its development and verification teams to satisfy it. You can identify three specific changes to a typical agile web application development process that would be required to comply with DO-178C DAL B.

2. Understand the ARINC 653 partitioned RTOS and IMA architecture — the execution model for modern commercial avionics software 4–5 hours

Skill: The real-time execution environment that replaces Linux/RTOS in commercial avionics

Why the field cares: ARINC 653 is the operating system standard for Integrated Modular Avionics (IMA) — the computing platform used on the A380, A350, B787, A220, and every modern commercial aircraft. Its execution model is fundamentally different from any RTOS a CS engineer is likely to have encountered: spatial partitioning (each application runs in its own protected memory space and cannot access another application's memory — achieved through an MMU configuration that the RTOS enforces, not through any application-layer sandbox), temporal partitioning (each application is allocated a fixed time slot in the cyclic executive schedule and receives exactly that time every frame — no more, no less, regardless of whether it needs it), and inter-partition communi-

4–5 hours

cation through typed ports (sampling ports for the most recent value, queuing ports for buffered message exchange — not shared memory, not sockets, not pipes). The CS engineer accustomed to Linux processes with dynamic resource allocation, preemptive scheduling, and socket-based IPC is operating in a fundamentally different programming model, and understanding this model before the interview — not discovering it during onboarding — is the preparation that avionics software employers look for.

Done signal: You can explain ARINC 653 spatial partitioning — what it prevents (one partition reading or corrupting another's memory), how it is implemented (MMU page table configuration by the OS, not application-layer memory protection), and why it is required for hosting multiple DAL levels on a single hardware platform. You can explain temporal partitioning — what it guarantees (each partition receives its allocated time every frame, with no interference from other partitions), why this is important for real-time scheduling analysis (the execution time of every partition is bounded and predictable), and what the consequence is for partition design (the partition must complete all required work within its time budget — CPU time cannot be borrowed from the next frame). You can describe the difference between a sampling port and a queuing port and explain which is appropriate for a flight parameter that is refreshed by the sensor 50 times per second.

4–5 hours

3. Understand worst-case execution time analysis — why average performance is irrelevant and what WCET requires from code structure

Skill: The real-time analysis methodology that replaces performance profiling for safety-critical software

Why the field cares: In the technology industry, software performance is measured by average execution time and tail latency percentiles. In safety-critical avionics, performance is measured by worst-case execution time — the maximum time any execution of a function can take, under any input, in any hardware state. The WCET must be demonstrated to be within the function's allocated time budget, and the demonstration must be traceable (measurement-based WCET analysis, static WCET analysis, or a hybrid approach). WCET analysis imposes specific code structure requirements: no dynamic dispatch (virtual functions in C++ are disallowed in some contexts because the dispatch table lookup adds unpredictable indirection), no data-dependent loops (loops whose iteration count depends on runtime data make static WCET analysis difficult or impossible — fixed-count loops are preferred), and no cache-dependent timing (instruction cache and data cache behaviour adds significant uncertainty to WCET estimates, and some avionics programs disable instruction caching entirely to achieve WCET predictability at the cost of average performance). The CS engineer who understands these constraints — and who can explain why a recursive Fibonacci function is not acceptable in a DO-178C context even if it is correct — has grasped the fundamental difference between software quality in the technology industry and software quality in safety-critical avionics.

Done signal: You can explain what WCET means and why it is the relevant performance metric for safety-critical real-time software (not average time, because the worst-case must always complete within the time budget — a single worst-case overrun can cause a timing fault that cascades through the entire cyclic schedule). You can describe why recursive functions are prohibited in many avionics coding standards (the stack depth cannot be bounded at compile time, making stack overflow a possible runtime failure without a guaranteed detection mechanism). You can explain how data-dependent loops complicate WCET analysis and what the typical avionics coding standard solution is (fixed-count loops with a maximum iteration count proven to be sufficient by analysis). You can describe at least one MISRA-C rule relevant to WCET analysis and explain what failure mode it prevents.

5–6 hours

4. Learn the verification and validation methodology for safety-critical software — requirements traceability, structural coverage, and independence

Skill: The V&V approach that replaces TDD and code coverage for safety-critical aerospace software

Why the field cares: Test-driven development — writing tests before writing code — is widely taught

5–6 hours

in CS programmes and widely practiced in the technology industry. In DO-178C, the equivalent concept exists but with a crucial difference: the tests are derived from the software requirements, not from the developer's intuition about what the code should do. Every test case must trace to one or more requirements it verifies, and every requirement must be verified by at least one test case — the traceability matrix is the formal evidence that the test suite is complete with respect to the requirements. Additionally, DO-178C requires demonstration of structural coverage — that the test suite exercises every line of code, every branch, and (at DAL A) every condition in every decision independently. A test suite that achieves 100% line coverage but does not achieve MC/DC may still leave critical uncovered code paths that represent potential failure modes. The independence requirement — that the verification activities are performed by personnel who did not perform the development — further separates aerospace V&V from the self-testing culture of agile development.

Done signal: You can describe the DO-178C requirements traceability matrix — what it traces (high-level requirements to low-level requirements, low-level requirements to source code, low-level requirements to test cases, test cases to test results), what it demonstrates (that every requirement is implemented and tested, that every test case is traceable to a requirement), and why it is a required configuration-controlled artifact. You can explain the difference between statement coverage, branch coverage, and MC/DC coverage using a specific code example with a two-condition Boolean expression, demonstrating which test inputs are needed for each level of coverage. You can explain why achieving 100% branch coverage does not guarantee detection of all logic errors and give one example of a logic error that branch coverage would miss but MC/DC would catch.

4–5 hours

5. Understand the aerospace systems safety methodology — how software fits into ARP4761/4754A

Skill: The safety engineering framework that determines what development assurance level your software must meet

Why the field cares: Software does not exist in isolation in aerospace — it implements functions that have safety consequences at the aircraft level. The Development Assurance Level assigned to a software function is derived from the severity of the failure condition of that function, determined through the ARP4761 system safety assessment process. The CS engineer who understands this derivation — who knows that the flight management system's VNAV function must be developed at DAL A because its failure (providing incorrect vertical guidance) would have Catastrophic consequences, while the cabin entertainment system can be developed at DAL E because its failure has No Safety Effect — understands why DO-178C exists and why its requirements scale with the DAL. Without this understanding, DO-178C appears as an arbitrary bureaucratic imposition. With it, DO-178C appears as a principled engineering response to the specific failure consequence of the software being developed — which is what it is.

Done signal: You can trace the safety analysis chain from aircraft-level failure condition to software DAL: starting with an aircraft-level Functional Hazard Assessment entry (loss of primary flight display, classified as Hazardous), through the Preliminary System Safety Assessment that assigns a probability target to the display system ($< 10^{-7}$ per flight hour for Hazardous), to the system architecture that allocates this probability across components, to the DAL B assignment for the display software. You can explain why the DAL assignment does not depend on the software's complexity — a simple function implementing a safety-critical feature requires the same DAL as a complex one implementing the same feature, because DAL is about the failure consequence, not the implementation difficulty.

8–10 hours

6. Learn C at the level required for safety-critical avionics software development

Skill: The primary programming language of safety-critical avionics — and why modern language choices do not apply

Why the field cares: Safety-critical avionics software is developed predominantly in C — specifically, C as constrained by MISRA-C (Motor Industry Software Reliability Association C coding guidelines),

8–10 hours

which prohibits dynamic memory allocation, restricts pointer arithmetic, mandates explicit error handling, and eliminates language features whose runtime behaviour is implementation-defined or undefined. The CS engineer trained primarily in Python, Java, JavaScript, or modern C++ has a language gap that they must close specifically — not by abandoning their existing languages (Python is extensively used in ground systems, data analysis, and non-safety-critical tool development in aerospace) but by developing the C fluency and MISRA-C awareness that safety-critical avionics roles require. The specific C skills most relevant to avionics: fixed-size integer types (`uint8_t`, `int16_t`, etc. — not `int`, which has implementation-defined size), bit manipulation for register-level hardware interfacing, static memory allocation patterns, and pointer arithmetic discipline.

Done signal: You can write a MISRA-C compliant C function that: uses only fixed-size integer types, performs no dynamic memory allocation, has no recursive calls, has all variables declared at the top of the function (C89 style, required by some avionics standards), has explicit return value checking for all function calls that can fail, and includes a comment identifying the DO-178C requirement it implements. You can identify at least five MISRA-C rules (from the MISRA-C:2012 guideline) and explain what undefined or implementation-defined behaviour each prevents. You understand the difference between C89, C99, and C11 and know which standard a specific avionics coding standard requires.

10–20 hours

7. Understand the physical domain of your target aerospace application — at depth, not at concept level

Skill: The domain physics that determines whether your software is producing meaningful outputs

Why the field cares: Every aerospace software system computes on physical quantities — inertial position and velocity from sensor measurements, radar backscatter from a distributed target field, surface reflectance from satellite imagery, fuel state from flow meter integrations. The CS engineer who treats these as abstract numerical inputs and outputs — who implements a Kalman filter without understanding what the state vector's physical units are, who processes GNSS pseudoranges without understanding what atmospheric delay does to them, who applies machine learning to satellite imagery without understanding the radiometric calibration that must precede any meaningful spectral analysis — is producing software whose correctness cannot be verified by the developer and whose errors will not be caught until the system is integrated with real hardware or real data. Domain physics is not optional context for aerospace software engineers — it is the first-level correctness criterion for any software that computes on physical measurements.

Done signal: For the specific aerospace domain you are targeting, you can describe the physical measurement that your software will process (inertial acceleration in m/s^2 , radar cross-section in dBsm , top-of-atmosphere radiance in $\text{W/m}^2/\text{sr}/\mu\text{m}$, aircraft position in WGS84 latitude/longitude/altitude), the primary physical effects that introduce errors into that measurement (accelerometer bias and scale factor for IMU, atmospheric propagation for GNSS, atmospheric scattering and absorption for optical EO), and what the consequence of an uncorrected error in that measurement would be for the software's output. You can explain how your code would be wrong if you treated the raw sensor output as a perfectly accurate physical quantity.

20–40 hours

8. Build a portfolio project in your target aerospace domain — demonstrating both software engineering quality and aerospace domain engagement

Skill: The most direct signal of genuine aerospace preparation that CS engineers can provide

Why the field cares: CS engineers are at an advantage in portfolio construction — they can build and document software projects more rapidly than engineers in hardware-intensive disciplines. This advantage must be used deliberately: not to build impressive software in isolation from aerospace context, but to build software that demonstrates aerospace domain engagement at technical depth. The most effective portfolio projects for CS engineers entering aerospace: a GNSS PVT solver in Python processing RINEX data and comparing results against known reference positions (for GNSS/navigation roles); a radar target detection algorithm implementing CFAR detection on

20–40 hours

recorded IQ data from a software-defined radio (for radar roles); a satellite imagery preprocessing pipeline in Python applying atmospheric correction and cloud masking to Sentinel-2 data (for EO data engineering roles); a MISRA-C compliant implementation of a digital filter in C with a WCET analysis and a DO-178C-style requirements traceability table (for avionics software roles); or a U-Space conflict detection and resolution algorithm processing ADS-B data from the OpenSky Network (for UTM/autonomy roles). Each of these produces something specific, technical, and documentable that directly demonstrates aerospace domain engagement.

Done signal: You have completed a portfolio project in your target aerospace domain that produces a specific, documented technical output. The documentation includes: the problem statement with its aerospace context (not just the algorithmic challenge), the design decisions and their aerospace-domain rationale, the validation approach (how you verified the outputs are physically correct, not just algorithmically correct), and the limitations (what the project does not do that a production aerospace system would require). You can present it in 10 minutes, spending equal time on the aerospace domain motivation and the technical implementation.

3–4 hours

9. Prepare behavioural answers that translate CS experience into aerospace-relevant terms — connecting software to its safety and physical consequence

Skill: The narrative translation that makes CS experience legible to aerospace interviewers

Why the field cares: Aerospace interviewers hiring CS-background candidates face the same interpretation challenge as those hiring from other disciplines: they cannot immediately assess whether experience building a recommendation system, an ML pipeline, or a distributed microservices architecture is relevant to their role. The CS engineer who can make this translation explicitly — “I built a real-time streaming data pipeline processing 50,000 events per second with sub-100ms end-to-end latency; in a UTM context, this would translate to handling conflict detection queries for 10,000 simultaneous aircraft operations with the latency requirement set by the aircraft’s minimum response time to a conflict advisory, which is approximately 30 seconds — orders of magnitude more lenient than my current pipeline but with regulatory requirements for availability and integrity that my current system does not have” — is demonstrating both technical competence and aerospace contextualisation simultaneously. The translation must be explicit; the interviewer will not make it for you.

Done signal: For each of four significant CS projects in your career history, you have prepared a translation narrative that: describes the project in CS/software engineering terms with specific technical metrics (throughput, latency, model accuracy, system availability), maps the key technical elements to their aerospace equivalents with explicit identification of what changes in the aerospace context (the latency requirement is different; the availability and integrity requirements are governed by a specific regulatory standard; the data source has specific physical properties that must be understood to validate the software’s output), and identifies the specific gap between your current system and what a production aerospace system would require.

4–5 hours

10. Understand software configuration management at the aerospace level — the SCM plan, the problem report system, and the change impact analysis

Skill: The configuration management rigour that DO-178C requires beyond standard software CM practice

Why the field cares: Most CS engineers use Git and consider their configuration management practice solid. DO-178C configuration management is substantially more demanding than any typical software CM practice, even at technology companies with strong engineering cultures. Every software artifact — not just source code, but also requirements documents, design documents, test procedures, test results, review records, and problem reports — is under formal configuration control with a Software Configuration Management Plan that specifies the CM procedures, the configuration item list, the baseline management approach, and the change control procedures. Every problem found during development or testing must be recorded in a formal Problem Report with a description, an impact assessment (does this problem affect other parts of the software? does it affect the test results

4–5 hours

already recorded?), a resolution, and a re-test record. Every change to the software baseline requires a documented impact analysis that traces the change through the requirements, code, and tests to identify what must be re-verified.

Done signal: You can describe what a DO-178C Software Configuration Item (SCI) is and give five examples of the types of artifacts that are under formal CM in a DO-178C compliant programme. You can describe the DO-178C Problem Report process — what triggers a problem report (any discrepancy between the software’s behaviour and its requirements, or between the verification results and the expected results), what the problem report must contain (problem description, severity, affected configuration items, planned corrective action, actual corrective action, re-test records), and how the problem report system interacts with the baseline change control process. You can explain what a software baseline is and what it means to formally baseline a configuration — specifically what becomes immutable and what the process is for making changes after baseline.

ROLE-SPECIFIC — TARGET AFTER FOUNDATIONAL TASKS

6–8 hours

11. For avionics software roles: implement a MISRA-C compliant embedded function with static analysis validation

Skill: The practical C coding competence that avionics software interviews assess

Why the field cares: Avionics software interviews frequently include a live or take-home coding exercise in C — not in Python, not in C++, but in C with MISRA-C constraints. The candidate who arrives having only recently reviewed C syntax but who has not written MISRA-C compliant code recently will struggle with the specific rules that most constrain the implementation: no implicit type conversions (all arithmetic must use explicit casts to maintain type traceability), no recursion, no dynamic memory, all variables initialised at declaration, all switch statements with a default clause, and no dead code (every code path must be reachable and intentional). Running MISRA-C static analysis on your code — using PC-lint, LDRA, Polyspace, or the open-source cppcheck with MISRA rules enabled — and correcting every reported violation is the most direct way to build MISRA-C coding competence before the interview.

Done signal: You have written, compiled, and statically analysed a MISRA-C compliant C function of at least 50 lines that implements a meaningful computation (a digital FIR filter, a PID controller, a CRC computation, or a data conversion routine) — using only static memory allocation, no recursion, fixed-size integer types throughout, and explicit error handling for all operations that can fail. You have run the code through at least one MISRA-C static analysis tool and resolved every mandatory and required rule violation. You can explain the rationale for at least five specific MISRA-C rules that appeared in your code.

4–5 hours

12. For ML and autonomous systems roles: understand the EASA AI roadmap and the emerging certification approach for ML-based functions

Skill: The regulatory frontier for machine learning in aviation — the domain where CS skills are most needed and least certificated

Why the field cares: Machine learning is entering aviation faster than the regulatory framework can certify it. EASA’s Artificial Intelligence Roadmap (published 2023, updated 2024) defines a phased approach to certifying ML-based functions: Level 1 (AI assistance to humans, no autonomous decision authority), Level 2 (AI performing functions with human monitoring and override capability), and Level 3 (fully autonomous AI performing safety functions without human oversight). The vast majority of currently operational AI in aviation is Level 1 (a neural network flagging anomalies for human review). Level 2 is beginning to be certified for specific functions (EASA’s concept paper on AI for aviation applications in 2023 covers Level 2 approaches). Level 3 does not have a certification framework yet. Understanding where the current regulatory boundary is — what ML applications are certifiable today, what the evidence requirements are, and what the near-term regulatory development looks like — is the AI regulatory literacy that CS engineers entering autonomous aviation,

4–5 hours

predictive maintenance, or AI-assisted avionics roles must have.

Done signal: You can describe EASA's AI roadmap — the three levels of autonomy and what each requires in terms of learning assurance, distribution awareness, and operational design domain. You can explain what the EASA EUROCAE WG-114/RTCA SC-216 joint committee has published on AI assurance for aviation. You can describe what a Level 1 ML application in aviation looks like — a specific example such as an anomaly detection system that flags maintenance issues for human review — and describe what evidence would be required to demonstrate its acceptability to EASA under the current regulatory approach.

6–8 hours

13. For EO and remote sensing data roles: implement a Sentinel-2 processing pipeline from download to surface reflectance product

Skill: The hands-on geospatial data engineering competence that distinguishes engaged EO candidates

Why the field cares: Earth observation data engineering is one of the most accessible and rapidly growing aerospace software domains for CS engineers — and the one where the gap between generic data engineering skills and domain-specific competence is most visible. The CS engineer who can build a scalable data pipeline but who has never processed a GeoTIFF, applied a radiometric scale factor, handled a coordinate reference system transformation, or applied a cloud mask to a multispectral image is missing the domain-specific data literacy that EO data engineering requires. Processing a complete Sentinel-2 scene from the Copernicus Data Space API through to a cloud-masked surface reflectance product with correct georeferencing — using rasterio, GDAL, and either the Sen2Cor atmospheric correction or a Python implementation of the 6S radiative transfer model — is the portfolio project that most directly demonstrates EO data engineering readiness.

Done signal: You have implemented a Python pipeline that: authenticates to the Copernicus Data Space Ecosystem API, searches for and downloads a cloud-free Sentinel-2 shadow band scene, applies the atmospheric correction to produce a surface reflectance product, applies the scene classification layer as a cloud and cloud-shadow mask, computes NDVI for the masked surface reflectance pixels, and writes the result as a georeferenced GeoTIFF with correct CRS metadata. You can explain every step, the physical meaning of each spectral band used, and the sources of uncertainty in the surface reflectance estimate.

6–8 hours

14. For UTM and ground segment roles: understand the U-Space regulatory framework and implement a simplified conflict detection algorithm

Skill: The airspace management regulatory and algorithmic knowledge for UTM software roles

Why the field cares: UTM software — the systems that manage the shared use of low-altitude airspace by UAVs, eVTOL air taxis, and other novel aircraft — is one of the most software-intensive roles in aerospace and one where CS background transfers most directly. But the specific regulatory framework (U-Space in Europe, UTM in the US), the specific data standards (NET-RID for network remote identification, the ASTM F3411 standard for remote ID message format, the OpenAPI specifications for USSP inter-communication), and the specific algorithmic challenges (conflict detection in 4D — three spatial dimensions plus time — for aircraft with uncertain trajectories) are aerospace domain knowledge that generic software engineering backgrounds do not provide. Implementing a simplified conflict detection algorithm — checking whether the 4D trajectory cylinders of two aircraft intersect, given their planned routes and timing, and computing the time and location of first potential conflict — is an accessible and directly relevant portfolio project that demonstrates both algorithmic competence and UTM domain engagement.

Done signal: You can describe the U-Space service architecture — the four service layers and what each provides operationally, the role of the Common Information Service, and how U-Space Service Providers interconnect. You can describe the NET-RID message format — what data fields it carries, what update rate is required, and why the broadcast and network remote ID modes have different requirements. You have implemented a simplified 4D conflict detection algorithm that takes two air-

craft trajectory plans (waypoint sequences with associated timing) and returns the time and location of first potential conflict within a specified separation volume.

6–8 hours

15. For cybersecurity roles: understand DO-326A/ED-202A and the ARINC 811 cybersecurity framework for avionics

5–6 hours

Skill: The aviation-specific cybersecurity regulatory framework that governs the security of certificated aircraft systems

Why the field cares: Aviation cybersecurity is a rapidly growing domain with specific regulatory requirements that have no direct equivalent in enterprise IT security. DO-326A (Airworthiness Security Process for Commercial Aircraft) / ED-202A (the EUROCAE equivalent) defines the security assessment process for airworthiness-relevant cybersecurity threats to certificated aircraft — specifically, threats to systems that, if compromised, could have safety consequences (loss of aircraft control, navigation system spoofing, flight data manipulation). ARINC 811 (Commercial Aircraft Information Security Concepts of Operation and Process Framework) defines the security architecture for commercial aviation information systems. The cybersecurity engineer from an enterprise IT background is experienced in threat modelling, penetration testing, and vulnerability assessment — but must understand that aviation cybersecurity threat assessment is governed by a specific regulatory process, that security requirements must be integrated with the aircraft's safety assessment, and that the avionics network architecture has specific characteristics (ARINC 664 AFDX with end-to-end integrity checking, physically separate networks for different data classification levels) that differ from enterprise IT network architectures.

Done signal: You can describe what DO-326A requires for the aircraft security risk assessment — the threat scenario identification, the security risk classification (based on the safety consequence of a successful attack and the likelihood of an attacker achieving it), the security objectives derived from the risk classification, and the means of compliance that demonstrate the security objectives are met. You can describe the ARINC 664 AFDX network architecture — the Virtual Links that provide deterministic, bounded-latency communication, the end-system integrity checking that prevents frame injection, and the network switch architecture that provides physical separation between different data classification levels. You can explain why a threat that allows an attacker to corrupt navigation data on the aircraft is a DO-326A concern regardless of how unlikely the attacker is to succeed — because the safety consequence of success is Catastrophic.

16. For autonomous systems roles: understand the current limits of autonomous operation approval and the detect-and-avoid requirement

4–5 hours

Skill: The regulatory boundary for autonomous aviation and what crossing it requires technically and regulatorily

Why the field cares: Every CS engineer with autonomous systems experience who targets the aviation autonomy domain must understand where the current regulatory boundary is — what level of autonomy is currently certifiable, under what conditions, and for what aircraft categories. The CS engineer who designs an autonomous decision-making system for an eVTOL without understanding that no current regulatory framework authorises fully autonomous commercial passenger transport is making a product that cannot enter service, regardless of its technical excellence. The current state: BVLOS drone operations are being certificated under specific frameworks (EASA UAS Category Specific with SAIL IV–VI, FAA Part 107 waiver for BVLOS) with technology requirements including detect-and-avoid per ASTM F3442 and remote identification per ASTM F3411. Autonomous passenger-carrying air taxis are being certified with pilots on board — the autonomous capability is a backup and a failure recovery tool, not the primary operational mode. Understanding where the boundary is — and what technical demonstration would be required to move it — is the regulatory intelligence that CS engineers in autonomy roles must have.

Done signal: You can describe the ASTM F3442 standard for detect-and-avoid — what it requires the DAA system to detect (intruder aircraft, obstacles, terrain), what the alerting timing requirement

4–5 hours

is (the remain well clear manoeuvre must be initiated with sufficient time for the aircraft to avoid the collision region), and what operational credit the system provides (BVLOS operation authorisation in controlled airspace for specific aircraft classes). You can describe what “well clear” means geometrically under ASTM F3442 and explain why this is the safety-critical output of the DAA system. You can describe the current regulatory status of single-pilot and zero-pilot commercial passenger operations in Europe and the US.

4–5 hours

17. Learn aerospace software architecture patterns — the IMA LRU, the functional chain from sensor to effector, and the role of software in FMA

Skill: The systems architecture knowledge that connects individual software components to the aircraft-level safety case

Why the field cares: The CS engineer who designs software as a set of services, APIs, and data flows is using the right architectural thinking — but the specific patterns and constraints of aerospace software architecture are different from web microservices or embedded firmware architectures. In IMA avionics, the software architecture is defined by the functional chains that connect sensor inputs to effector outputs through a series of computing functions allocated to ARINC 653 partitions on one or more IMA modules. The latency of each functional chain (the time from sensor measurement to effector command) must be within the specified control loop requirements, and the latency must be analysed and demonstrated, not measured at average conditions. Every software function in the functional chain contributes to the failure mode analysis: a software function that can produce an incorrect output contributes to the failure condition analysis of the functional chain, and its development assurance level is set accordingly. Understanding this functional chain architecture — and being able to trace from a specific pilot control input through the software processing chain to the aircraft response — is the systems architecture knowledge that distinguishes a software architect who can contribute to avionics system design from one who can only implement within a pre-specified software architecture.

Done signal: You can describe what an avionics functional chain is — the sequence of computational steps from sensor input through processing to actuator output — and sketch a simplified functional chain for a flight control function (elevator command from pilot stick input, through flight control law computation, to elevator actuator position command). You can identify where each step in the chain contributes to the functional chain latency and explain how the total latency is analysed against the control loop’s timing requirement. You can explain how the failure mode analysis of the functional chain translates to DAL assignments for each software component in the chain.

4–5 hours

18. Build aerospace-specific testing and validation competence — HIL testing, simulation-based verification, and ground testing for flight software

Skill: The testing infrastructure and methodology for aerospace software, which extends well beyond unit and integration testing

Why the field cares: The testing of avionics and flight software goes well beyond the unit testing and integration testing that CS programmes teach. Hardware-in-the-loop (HIL) testing — running the actual flight software against a simulated aircraft model in real time, with real sensor hardware and real actuator hardware in the loop — is the primary validation environment for flight-critical avionics. The HIL test bench reproduces the aircraft’s electrical interfaces, sensor signal ranges, actuator dynamics, and communication bus timing, allowing the software to be exercised in conditions that are representative of its operational environment before the aircraft is built. Understanding what HIL testing provides (confidence that the software behaves correctly with real hardware interfaces and realistic signal characteristics, which simulation cannot fully replicate), what its limitations are (the simulation model is not the actual aircraft — aerodynamic model errors, sensor model errors, and actuator model errors all produce test results that may differ from the actual aircraft’s behaviour), and how the HIL test results contribute to the DO-178C verification evidence (as integration tests at the system level) are the advanced testing knowledge items that aerospace software engineers with

4–5 hours

product development responsibility must have.

Done signal: You can explain what a HIL test bench is, what hardware and software components it comprises, and what it tests that software-only simulation cannot replicate. You can describe one specific failure mode that HIL testing is specifically able to detect (a timing failure caused by the actual sensor's interrupt latency being longer than the software's assumed maximum interrupt service time) that software-only unit testing would not catch. You can explain how the HIL test results fit into the DO-178C evidence — specifically, what level of verification credit they provide and what additional verification is still required after HIL testing.

POSITIONING AND PRESENTATION

2–3 hours

19. Prepare a specific answer to “Why aerospace — and what from your CS background makes you ready for this role?”

Skill: The transition motivation and positioning answer that every aerospace CS interview opens with

Why the field cares: Aerospace interviewers hiring CS-background candidates have two specific concerns beyond those for ME or EE candidates. First: does this person understand that aerospace software development is fundamentally different from the software development culture they are coming from, and are they genuinely willing to adapt — or will they be frustrated by the constraints and try to introduce practices that work in their current context but are incompatible with DO-178C compliance? Second: is their interest in aerospace technically grounded, or are they attracted by the glamour of aerospace without genuinely understanding the engineering challenges? The answer must address both. For the first concern: specifically acknowledging the key differences (no dynamic allocation, requirements-driven development, MC/DC coverage, independence) and explaining why you find these constraints intellectually honest rather than bureaucratically obstructive — because they are principled engineering responses to the specific failure consequences of safety-critical software. For the second concern: naming the specific aerospace technical problem that most closely connects to your CS expertise and explaining why it is a harder or more interesting version of a problem you have already engaged with.

Done signal: You have a 2-minute answer that: acknowledges the specific ways your current software development practice would need to adapt for the target aerospace role (naming the constraint and explaining why it exists, not just that it exists); names the specific aerospace technical problem most connected to your CS expertise and explains why your background makes you specifically relevant; demonstrates that you have done meaningful preparation work (the portfolio project, the domain study, the DO-178C engagement); and explains why the consequence of the work — the specific safety or operational outcome that depends on the software being correct — is what motivates the adaptation rather than the fact that the employer requires it.

4–6 hours

20. Map your CS specialisation to the specific aerospace employers and roles where it is most competitive, accounting for the clearance landscape

Skill: The job market intelligence that focuses preparation effort on the highest-probability opportunities

Why the field cares: The aerospace software job market for CS engineers is both large and highly stratified. Some roles — Earth observation data engineering, UTM software development, ground segment software — are effectively software engineering roles in an aerospace context, with relatively modest domain knowledge requirements. Others — avionics software development at DO-178C DAL A, flight control law implementation, autonomous flight safety case — require both deep domain knowledge and a development methodology that is fundamentally different from commercial software practice. Many of the most technically interesting roles in radar, EW, and space intelligence are clearance-gated, with nationality requirements that may or may not match the candidate's situation. Mapping your CS specialisation to the roles where both your technical background and

4–6 hours

your regulatory preparation are strongest — and then identifying the specific employers, the specific teams within those employers, and the specific vacancies that match — is the targeting exercise that makes the transition job search efficient rather than diffuse.

Done signal: You have identified at least five specific employers where your CS specialisation is directly relevant to open or recurring roles — distinguishing between commercial software-intensive aerospace roles (where your CS background transfers most directly), safety-critical avionics software roles (where DO-178C preparation is required), and clearance-gated defence aerospace software roles (where clearance eligibility is the first filter). You have researched the specific open roles at each employer, understood what technical topics are most likely to be assessed, and prioritised your preparation effort accordingly. You have at least two active applications, each with a portfolio project specifically relevant to the target role's domain.

SUMMARY REFERENCE TABLE

#	Task Description
1	DO-178C at implementation depth — no malloc, requirements-driven, MC/DC, independent
All avionics targets / Do first	
2	ARINC 653 partitioned RTOS — spatial/temporal partitioning, ports, scheduling m
Avionics software	
3	Worst-case execution time analysis — why average performance is irrelevant, MISRA-C i
Avionics software	
4	Requirements traceability and structural coverage — V&V methodology replacing
Avionics software / All	
5	ARP4761/4754A safety methodology — how DAL is derived from failure condition s
All	
6	C at MISRA-C level — fixed types, no dynamic allocation, no recursion, static anal
Avionics software	
7	Physical domain of target application — what the software computes on, not just nu
All	
8	Portfolio project in target domain — aerospace context, physical correctness, docum
All / Critical	
9	STAR story translation — CS experience mapped to aerospace safety and domain cons
All	
10	Aerospace CM at DO-178C level — SCM plan, problem reports, change impact ana
Avionics software	
11	MISRA-C compliant C function with static analysis validation
Avionics software	
12	EASA AI roadmap and ML certification approach — Level 1/2/3, EUROCAE WG
ML / autonomous systems	
13	Sentinel-2 processing pipeline — download, atmospheric correction, cloud mask, N
EO data engineering	
14	U-Space framework and 4D conflict detection algorithm implementation
UTM / ground segment	
15	DO-326A/ED-202A and ARINC 811 — aviation cybersecurity regulatory framew
Cybersecurity	
16	Autonomous operation regulatory boundary — BVLOS, DAA, SC-VTOL-01, current
Autonomous systems	
17	Aerospace software architecture — functional chains, IMA LRU, software in FMI
Avionics / all	
18	HIL testing and simulation-based verification — what it tests that unit testing can

#	Task Description
Avionics / flight software	
19	“Why aerospace — what specifically transfers?” — 2-minute positioned answer
All	
20	Employer mapping accounting for clearance — avionics / EO / UTM / defence t
All	

THE STRUCTURAL REALITY OF THE CS-TO-AEROSPACE TRANSITION

The computer science engineer entering aerospace is doing something that is simultaneously easier and harder than either the EE or the ME transition. Easier, because the demand for CS skills in aerospace has never been higher — the sector's most urgent hiring needs in 2025 include autonomous systems engineers, Earth observation ML scientists, avionics software developers, cybersecurity engineers, and data platform architects, all of which are fundamentally CS roles in an aerospace context. Harder, because the gap between how software is developed in the technology industry and how it is developed in safety-critical aerospace is the largest cultural and methodological gap of any discipline transition in this series.

The specific character of this gap deserves emphasis. It is not a gap in technical skill — the CS engineer is often technically stronger than the traditional aerospace software engineer in algorithms, system design, and modern programming practices. It is a gap in what software correctness means. In the technology industry, software is correct if it passes its tests, deploys without errors, and meets its performance targets. In safety-critical avionics, software is correct only if it has been developed through a process that guarantees — not demonstrates, but guarantees — that every specified requirement has been implemented, every implementation decision has been verified against its requirements by an independent verifier, and every code path has been exercised by a test case specifically designed to exercise it. The process guarantee is the product, not just the code.

This distinction — between demonstrating correctness through test results and guaranteeing it through process — is the conceptual shift that the CS-to-aerospace transition requires. The CS engineer who has internalised this will find DO-178C not as a constraint imposed from outside but as an intellectually coherent engineering response to a specific failure domain: you cannot fully test a safety-critical system into correctness at any practical cost, so you must engineer the development process such that the output is provably correct by construction. The CS engineer who sees this as principled engineering — who finds the formal requirements traceability, the structural coverage analysis, and the independence requirements interesting rather than obstructive — is the one who will thrive in aerospace software development. The one who treats it as compliance overhead will struggle, and the struggle will be visible in every interaction with the certification team.

The opportunity for the CS engineer who makes this transition correctly is genuine: they bring technical capabilities — algorithmic sophistication, ML competence, distributed systems architecture, modern software engineering discipline — that aerospace has historically lacked, into a sector whose products are among the most consequential that engineering produces. The aircraft that lands safely at the end of a twelve-hour transoceanic flight does so because dozens of software systems computed correctly for every one of the forty-three thousand seconds of the flight. The CS engineer who contributed to any one of those systems — and who developed it correctly enough that it has never caused a flight safety event — has done engineering whose consequence is real, measurable, and permanent in the most meaningful sense.

BREAK INTO AEROSPACE
